



Universidade do Minho
Escola de Engenharia

12/10/2025

Universidade do Minho - Escola de Engenharia

Comunicações por Computador - TP2

Gonçalo Castro
a107337

Rita Cunha
a104261

Pedro Figueiredo
a104354

Índice

1 . Introdução	3
2 . Arquitetura Geral do Sistema	3
2.1 Protocolo MissionLink (sobre UDP)	4
2.1.1 Design e Arquitetura do Protocolo	4
2.1.2 Formato das Mensagens MissionLink	5
2.1.3 Funcionalidades do MissionLink	6
2.1.4 Mecanismos de Controlo Aplicados	7
2.2 Protocolo TelemetryStream (sobre TCP)	8
2.2.1 Design e Arquitetura	8
2.2.2 Formato das Mensagens de Telemetria	8
2.2.3 Funcionalidades do TelemetryStream	9
2.2.4 Mecanismos de Controlo Aplicados	9
2.3 API de Observação	10
2.3.1 Formato Geral das Respostas	10
2.3.2 Endpoints Principais	10
2.4 Eventos WebSockets / SSE	11
2.5 Ground Control	12
2.5.1 Funcionalidades do Ground Control	12
3 . Testes Realizados	13
3.1 Metodologia	13
3.2 Resultados	13
4 . Conclusões Finais	15
4.1 Melhorias Futuras	15

Lista de Figuras

Figura 1 Topologia	4
Figura 2 Diagrama temporal do fluxo de criação e execução de uma missão	5
Figura 3 1º Parte da Página Web do Ground Control	12
Figura 4 2º Parte da Página Web do Ground Control	13
Figura 5 Execução do teste 5 com TIMEOUT = 2s e MAX_RETRIES = 3	14
Figura 6 Execução do teste 5 com TIMEOUT = 0.1s e MAX_RETRIES = 3	14

1 . Introdução

Atualmente, a comunicação de dados assume um papel decisivo em praticamente todos os domínios tecnológicos. A crescente dependência de sistemas inteligentes e autónomos, particularmente em cenários espaciais ou de exploração remota, exige soluções de comunicação robustas, eficientes e resistentes a falhas. O presente relatório descreve o desenvolvimento de um sistema distribuído composto por uma Nave-Mãe, vários *Rovers* e um nó de Ground Control. O objetivo central consiste na construção de dois protocolos aplicacionais, **MissionLink (ML) sobre UDP** e **TelemetryStream (TS) sobre TCP**, capazes de garantir, respetivamente, operações fiáveis de missões e a transmissão periódica de telemetria de estado. Complementarmente, a Nave-Mãe expõe uma API de Observação, consumida pelo Ground Control, que permite monitorizar em tempo real o estado dos rovers e das missões.

As decisões de *design* tomadas resultam diretamente das exigências do enunciado, que impõe a necessidade de fiabilidade sobre UDP, capacidade de operar com perda e latência, suporte para múltiplos rovers em paralelo e integração simples através de uma API estruturada. Todas as componentes foram desenvolvidas privilegiando clareza, robustez e comportamento determinístico em cenários adversos, refletindo os desafios de comunicações espaciais simuladas.

2 . Arquitetura Geral do Sistema

A arquitetura do sistema é composta por três entidades principais: a Nave-Mãe, os Rovers e o Ground Control, interligados através de uma rede simulada no CORE. A Nave-Mãe atua como um centro de coordenação, então recebe telemetria, atribui missões e disponibiliza o estado global através de uma API. Os rovers, por sua vez, executam missões localmente, simulam movimento e enviam informação sobre o seu estado. O Ground Control consulta a API da Nave-Mãe e apresenta ao utilizador uma visão agregada do sistema, permitindo também a emissão de comandos de gestão de missões.

A inclusão de 3 *routers* como **sat (1 até 3)** simulam uma rede de satélites intermédios no CORE que permitem testar o sistema sob diferentes condições de comunicação, verificando o comportamento dos mecanismos de fiabilidade implementados. O router n1 serve também para distanciar os dois nós que separa (Nave-Mãe e GroundControl). Esta rede de satélites, em formato de malha parcial, assegura também redundância nas opções de caminho alternativos em caso de falha

O *Switch* nº1 (nomeado sw1) foi inserido pelo facto de garantir que a Nave-Mãe(MotherShip) possui apenas um IP do lado da sub-rede dos rovers, assim facilitando a configuração das comunicações com os mesmos. De realçar que a representação do Ponto de Carregamento é apenas visual no ambiente CORE, de modo a fixar a sua posição nas coordenadas $(x,y) = (50,50)$.

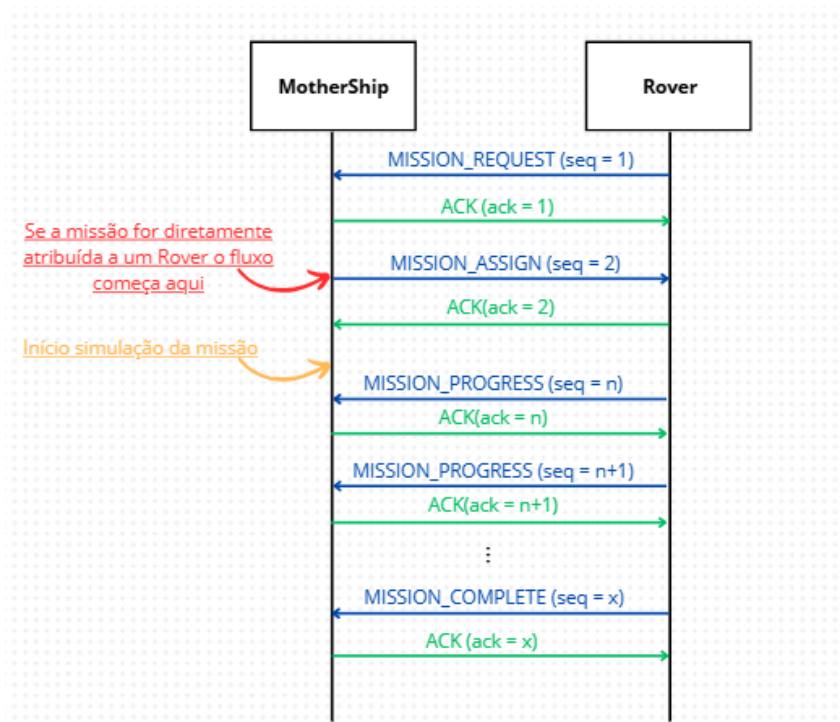


Figura 2: Diagrama temporal do fluxo de criação e execução de uma missão

2.1.2 Formato das Mensagens MissionLink

As mensagens MissionLink são binárias e têm sempre o mesmo tamanho (**150 bytes**). São constituídas por três partes: cabeçalho, payload e um campo checksum.

1. Cabeçalho (9 bytes)

Campo	Tamanho	Função
type	1 byte	Identifica o tipo da mensagem (HELLO, ASSIGN, PROGRESS, COMPLETE, ...)
seq	4 bytes	Número de sequência enviado pelo emissor
ack	4 bytes	Número de sequência reconhecido

Tabela 1: Estrutura de uma mensagem MissionLink

O cabeçalho contém toda a informação necessária ao controlo de fiabilidade e gestão da comunicação.

2. Payload

O payload descreve os parâmetros necessários para a execução de uma missão atribuída ao rover e alguns campos de estado associados. Tem tamanho fixo de 140 bytes e inclui:

Campo	Tipo/Tamanho	Descrição
mission_id	8 bytes	Identificador da missão
rover_id	8 bytes	Identificação do rover
area_x1, area_y1, area_x2, area_y2	16 bytes	Coordenadas da área da missão
tasks	64 bytes	Tarefa atribuída
duration	4 bytes	Duração máxima
progress_period	4 bytes	Intervalo entre envios de progresso
progress	4 bytes	Progresso atual (%)
battery	1 byte	Nível de bateria do rover no momento da emissão da mensagem
position(x,y,z)	12 bytes	Coordenadas tridimensionais estimadas do rover aquando da emissão da mensagem
padding	—	Preenchimento até atingir 140 bytes, permitindo futuras extensões sem alterar o formato da mensagem.

Tabela 2: Estrutura do payload

O uso de tamanho fixo reduz a complexidade e previne ambiguidades.

3. Checksum (1 byte)

Um byte final que contém a soma modular de todos os restantes campos da mensagem. Permite detetar corrupção antes do processamento.

2.1.3 Funcionalidades do MissionLink

O MissionLink implementa um conjunto de funcionalidades necessárias à coordenação das missões entre a Nave-Mãe e os Rovers.

- **“Hello” para Reconhecimento de Estado:** Quando um rover se conecta à Nave-Mãe, ele envia uma mensagem do tipo HELLO para se identificar e reconhecer o estado atual. Este tipo de mensagem é fundamental caso se deseje enviar diretamente uma missão ao rover, permitindo que a Nave-Mãe conheça o endereço completo do rover e a sua disponibilidade sem que o Rover tenha de enviar uma mensagem primeiro (UDP não possui *handshake* formal).
- **Requisitar Missão:** Se o rover não tiver uma missão atribuída, ele pode enviar uma mensagem **MISSION_REQUEST** para a Nave-Mãe solicitando uma missão. Caso não haja missões em espera (numa fila de espera na MotherShip) para atribuição, o rover será adicionado à lista de “waiting_rovers” da Nave-Mãe, aguardando que uma missão seja criada para ser atribuída automaticamente.
- **Criar e Atribuir Missão:** Quando uma missão é atribuída ao rover pela Nave-Mãe, este recebe uma mensagem **MISSION_ASSIGN**. Após a recepção e validação da missão, o rover envia uma mensagem **MISSION_ACKNOWLEDGMENT** para confirmar a atribuição. Nesse momento, o rover muda de estado IDLE para IN_MISSION e inicia a execução da missão.
- **Cancelar Missão:** A Nave-Mãe pode enviar uma mensagem **MISSION_CANCEL** para o rover caso precise de interromper uma missão em andamento. Quando o rover recebe esta mensagem, o mesmo

deve interromper imediatamente as suas atividades e voltar ao estado IDLE. A missão fica marcada como cancelada.

- **Abortar Missão:** Em determinadas situações, rover pode enviar uma mensagem **MISSION_ABORT** para indicar que não consegue continuar a missão. Isso pode ocorrer por vários motivos, incluindo: falhas internas, condições adversas, ou erro no sistema. Nesse caso, o rover interrompe a missão e passa para o estado IDLE (ou em casos específicos passa para o estado ERROR).
- **Pausar Missão:** A mensagem **MISSION_PAUSED** indica que a missão foi temporariamente suspensa. Esta pausa pode ocorrer devido a bateria insuficiente (abaixo de 20%), em que o rover entra automaticamente em modo ECO para poupar energia e dirigir-se ao posto de carregamento; ou devido a sobreaquecimento, onde o rover interrompe a missão para evitar danos térmicos, retomando as atividades somente após a temperatura retornar a níveis seguros.
- **Completar Missão:** Quando o rover conclui uma missão com sucesso, ele envia uma mensagem **MISSION_COMPLETE** à Nave-Mãe. Nesse momento, o rover volta ao estado IDLE, indicando que a missão foi finalizada e que está disponível para receber novas tarefas.
- **Controlo de Bateria e Carregamento:** Quando a bateria do rover atinge níveis abaixo de 20%, o rover pausa a missão automaticamente e entra em “modo ECO” para poupar energia. Posteriormente, dirige-se ao posto fixo de carregamento, deslocando-se a uma velocidade reduzida para preservar a bateria. Quando chega ao posto, o rover começa a carregar incrementalmente até 100%, retomando depois a missão dirigindo-se novamente para a área da missão com a bateria carregada.
- **Controlo de Temperatura:** Caso o rover atinja 100°C durante a execução da missão, o sistema pausa a missão automaticamente e o estado do rover muda para IDLE. O rover só retoma a missão após a temperatura descer para valores abaixo de 30°C, garantindo a integridade do sistema e a segurança das operações.

2.1.4 Mecanismos de Controlo Aplicados

A fiabilidade do MissionLink é garantida por mecanismos implementados diretamente no protocolo, visto que o UDP não oferece garantias de entrega. Os principais mecanismos incluem:

1. **Número de Sequência (seq):** Cada mensagem inclui um número de sequência único, permitindo que o recetor identifique duplicados ou pacotes fora de ordem.
2. **Acknowledgments (ACKs):** As mensagens exigem uma confirmação de receção. O emissor aguarda um ACK com o mesmo número de sequência, e a falta deste dentro de um tempo limite é interpretada como falha de entrega, consoante os valores configurados de tempo de timeout e máximo de tentativas (que iremos explorar melhor na secção de Testes).
3. **Tabela de Mensagens Pendentes:** Cada entidade mantém uma tabela de mensagens enviadas aguardando confirmação, com um temporizador associado a cada entrada. Se o temporizador expirar sem que o ACK tenha sido recebido, uma retransmissão é acionada.
4. **Retransmissões Automáticas:** Quando um ACK não é recebido dentro do tempo limite, o protocolo aciona uma retransmissão automática da mensagem. O número de tentativas é limitado para evitar retransmissões excessivas.
5. **Checksum:** Cada mensagem contém um checksum que verifica a integridade dos dados durante o transporte. Se a verificação falhar, a mensagem é descartada.

6. Detecção de Duplicados: O sistema garante que mensagens duplicadas não sejam processadas. Isso evita que pacotes reenviados tardiamente alterem o estado da missão.

7. Reliability Manager: O Reliability Manager, localizado no ficheiro `reliability.py`, gere toda esta lógica de fiabilidade, incluindo o envio de ACKs, retransmissões e verificação da integridade das mensagens.

2.2 Protocolo TelemetryStream (sobre TCP)

O TelemetryStream é o protocolo responsável pela comunicação contínua de dados de telemetria entre a Nave-Mãe e os rovers. Ao contrário do MissionLink, que utiliza UDP, o TelemetryStream funciona sobre TCP, oferecendo maior fiabilidade na entrega e ordenação das mensagens. Este protocolo foi projetado para enviar dados periódicos de telemetria, como posição, temperatura e *status* dos sensores do Rover, em tempo real.. A comunicação é baseada em conexões TCP persistentes, com cada rover mantendo uma conexão dedicada à Nave-Mãe, garantindo que os dados de telemetria sejam enviados em intervalos regulares.

2.2.1 Design e Arquitetura

A arquitetura do TelemetryStream baseia-se num modelo cliente-servidor. Cada rover atua como cliente, estabelecendo uma ligação TCP para a Nave-Mãe, enquanto a Nave-Mãe atua como servidor, e escuta numa porta fixa e mantendo uma sessão independente para cada rover. O design do protocolo é baseado em três princípios principais: o fluxo contínuo de dados, com o envio periódico de telemetria, o uso de mensagens binárias de tamanho fixo, garantindo parsing eficiente, e a monitorização da ligação, permitindo a deteção rápida de falhas ou desconexões.

2.2.2 Formato das Mensagens de Telemetria

A mensagem de telemetria do TelemetryStream tem um formato fixo de 52 bytes, composto por vários campos essenciais. A tabela a seguir descreve a estrutura completa da mensagem de telemetria.

Cada TelemetryMessage contém:

Campo	Tipo/Tamanho	Descrição
magic	2 bytes	Identificador constante da mensagem
SIZE	2 bytes	Tamanho fixo da mensagem (52 bytes)
rover_id	8 bytes	Identificação do rover
mission_id	8 bytes	Identificador da missão
state	1 byte	Estado do rover (ex: IDLE, IN_MISSION, ERROR).
battery	1 byte	% de bateria
temperature	4 bytes	Temperatura interna
position (x, y, z)	12 bytes	Posição atual
speed (x, y, z)	12 bytes	Velocidade atual
sensor_flag	1 byte	Validação dos sensores (1 bit/sensor)
checksum	1 byte	Validação de integridade

Tabela 3: Estrutura da Mensagem de Telemetria

Se uma mensagem não respeitar o tamanho esperado ou apresentar um checksum inválido, é descartada.

2.2.3 Funcionalidades do TelemetryStream

- **Envio Periódico de Telemetria:** O rover envia dados periodicamente, permitindo à Nave-Mãe monitorizar o estado atual do rover e garantir que os dados são atualizados em tempo real.
- **Verificação de Sensores:** O TelemetryStream verifica constantemente os sensores do rover, incluindo temperatura interna, nível de bateria, pressão do ar nos pneus e estado da antena. Se algum sensor falhar, o rover entra em estado “ERROR”, e a telemetria é considerada inválida. Sensores de temperatura e bateria têm faixas de operação específicas: a temperatura deve estar entre -20°C e 150°C, e a bateria entre 0% e 100%, valores fora destes intervalos são considerados como falhas de leitura nos sensores.
- **Gestão de Erros:** Se ocorrer uma falha nos sensores ou nos dados, o TelemetryStream detecta rapidamente e coloca o rover em estado de erro, garantindo que a Nave-Mãe só receba dados válidos.

2.2.4 Mecanismos de Controlo Aplicados

Apesar de o TCP assegurar entrega fiável, o TelemetryStream integra mecanismos próprios ao nível da aplicação para garantir consistência e deteção de anomalias:

- **Verificação de integridade:**

Cada mensagem inclui um checksum que permite confirmar que os dados não foram corrompidos durante a transmissão antes de serem processados.

- **Periodicidade fixa de envio:**

Os rovers enviam telemetria em intervalos regulares e configuráveis. A Nave-Mãe monitoriza estes intervalos para detetar falhas.

- **Validação estrutural:** Como cada mensagem tem o mesmo tamanho, qualquer desvio na leitura, causado por interrupções parciais ou erros de ligação, é detectado automaticamente.

O conjunto destes mecanismos permite ao TelemetryStream fornecer uma monitorização contínua, fiável e eficiente, complementando o MissionLink e assegurando que o sistema mantém sempre uma visão coerente dos rovers.

2.3 API de Observação

A Nave-Mãe expõe uma API de Observação que disponibiliza a informação agregada relativa aos rovers, missões e telemetria. Esta API funciona como camada de abstração sobre os protocolos internos MissionLink (UDP) e TelemetryStream (TCP), apresentando o estado global do sistema através de mensagens JSON simples e independentes do formato binário utilizado internamente.

A API é acessível através de HTTP (REST) e suporta ainda eventos em tempo real através de WebSocket. Esta combinação permite recolher snapshots periódicos do sistema, bem como receber atualizações imediatas sempre que chegam novos dados de telemetria ou mudanças de estado das missões.

2.3.1 Formato Geral das Respostas

Independentemente do *endpoint*, todas as respostas seguem o mesmo modelo lógico, baseado na estrutura interna APIEvent. Esta estrutura permite encapsular qualquer tipo de informação dentro de um formato previsível.

Campo	Descrição
event	Tipo do evento devolvido (ex.: estado do rover, lista de missões, telemetria).
timestamp	Momento em que o evento foi gerado.
rover_id	Identificador do rover associado, quando aplicável.
mission_id	Identificador da missão associada, quando aplicável.
data	Objeto contendo a informação específica do endpoint.

Tabela 4: Estrutura base de uma resposta da API

O campo data adapta-se ao contexto de cada pedido, mantendo assim um modelo externo estável e previsível para todos os consumidores da API.

2.3.2 Endpoints Principais

A API está organizada em três grupos principais: Rovers, Missões e Telemetria. Seguem-se os endpoints relevantes, apresentados de forma resumida.

1. Rovers

Estes endpoints permitem obter o estado atual ou histórico de um rover.

Método	Endpoint	Descrição
GET	/rovers	Devolve o estado atual de todos os rovers conhecidos.
GET	/rovers/<rover_id>	Devolve o estado mais recente de um rover específico.

Tabela 5: Endpoints relacionados com Rovers

2. Missões

Estes endpoints permitem consultar e manipular missões, incluindo o envio e cancelamento.

Método	Endpoint	Descrição
GET	/missions	Lista todas as missões registradas.
GET	/missions/<mission_id>	Devolve detalhes de uma missão específica.
POST	/send-mission	Envia uma nova missão para um rover.
POST	/missions/<mission_id>/cancel	Cancela uma missão ativa.

Tabela 6: Endpoints relacionados com Missões

3. Telemetria

Estes endpoints fornecem acesso à informação de telemetria agregada pela Nave-Mãe.

Método	Endpoint	Descrição
GET	/telemetry/latest/limit=?	Devolve um conjunto de amostras recentes da telemetria desse rover.

Tabela 7: Endpoints relacionados com Telemetria

2.4 Eventos WebSockets / SSE

Além dos endpoints HTTP, a API pode emitir eventos em tempo real utilizando WebSockets através da biblioteca do Python chamada **Flask-SocketIO**. Estes eventos permitem atualizar o estado dos rovers e das missões sem necessidade de polling contínuo.

Os eventos partilham a mesma estrutura JSON usada nas respostas HTTP.

Evento	Código (event_code)	Descrição
telemetry	1	Nova telemetria recebida.
low_battery	2	Atualização do progresso de uma missão.
state_change	3	Atualização do estado do rover
mission_assigned	4	Missão enviada com sucesso.
mission_progress	5	Progresso da missão
mission_completed	6	Missão concluída
mission_aborted	7	Cancelamento da missão
rover_snapshot	10	Snapshot do estado instantâneo dos rovers
mission_snapshot	11	Snapshot do estado instantâneo das missões

Tabela 8: Eventos WebSockets

2.5 Ground Control

O Ground Control é uma interface essencial para o acompanhamento e gestão das missões dos rovers. Ele fornece uma visão em tempo real do status e da localização dos rovers, bem como a capacidade de interagir com as missões em andamento, como atribuição, atualização e monitoramento do progresso das tarefas.

A comunicação entre o Ground Control e a Nave-Mãe ocorre por uma API, que pode ser consumida usando HTTP e WebSockets, permitindo que o Ground Control receba dados de telemetria e atualizações de status em tempo real.

2.5.1 Funcionalidades do Ground Control

O Ground Control é a interface que permite monitorar e gerir as missões dos rovers em tempo real. Ele exibe o estado dos rovers, incluindo posição, bateria e estado, e permite o envio e atualização de missões com parâmetros como na Nave-Mãe. O mapa interativo exibe a localização dos rovers em tempo real, facilitando a visualização do progresso das missões.

A interface também permite que o Ground Control acompanhe o progresso das missões e reaja a falhas ou erros nos rovers. Esta interface foi desenvolvida de uma forma simplista com HTML(estrutura), CSS(estilos) e JavaScript(interatividade).

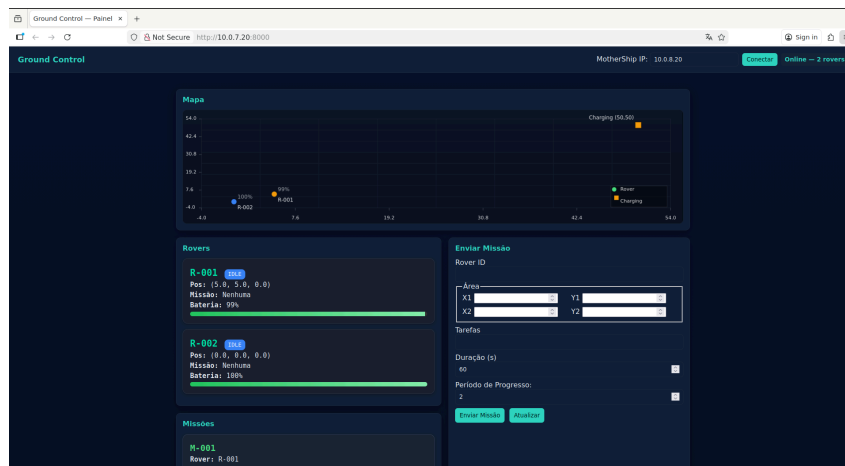


Figura 3: 1º Parte da Página Web do Ground Control

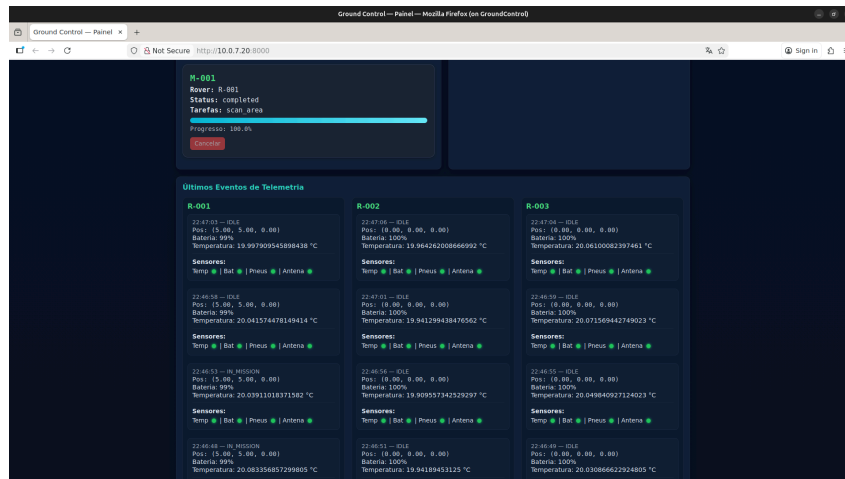


Figura 4: 2º Parte da Página Web do Ground Control

3 . Testes Realizados

3.1 Metodologia

A topologia foi concebida de tal forma que os três rovers pudessem designar diferentes possíveis estados da rede (configurada através da especificação dos campos da ligação **Bandwith**, **Delay**, **Jitter** e **Loss**) numa possível implementação real, tendo-se chegado à seguinte especificação geral: o Rover nº1 é o caminho “seguro”, pois possui uma conexão com o *sat1* (e posteriormente com a MotherShip) que foi configurada com um valor mínimo de latência, perdas e *jitter* para conduzir a uma conexão mais fiável para os testes durante todo o decorrer do trabalho inicial de implementação; o Rover nº2 opõe-se ao anterior, de modo a testar com valores um pouco mais exigente, de modo a que conseguimos testar se o nosso sistema de controlo de falhas previne este tipo de acontecimentos adversos; e por fim o Rover nº3 é um desafio à nossa solução pois permite testar a comunicação mais desafiante que nós implementamos, de forma a perceber se os nossos protocolos são suficientemente robustos e resilientes a estas perturbações externas.

Nota: Toda a topologia pode ser iniciada automaticamente, desde que a pasta que contém os ficheiros do projeto esteja dentro da pasta acessível pela rede interna do Docker **volumes/**. Este *script* inicia-se com o comando:

```
./start.sh
```

3.2 Resultados

Os seguintes testes foram executados automaticamente recorrendo ao comando:

```
./run_tests.sh 5
```

Nota: O número 5 indica o número do teste a executar segundo a ordem indicada no ficheiro *mission_scenarios.yaml* da pasta *tests/* que contém todos os testes concebidos no âmbito deste projeto.

De constatar que o seguinte teste foi executado com as seguintes configurações do módulo **Reliability**: **RELIABILITY_TIMEOUT** = 2.0 (segundos) e **RELIABILITY_MAX_RETRIES** = 3, ambas definidas no ficheiro *protocol_config.py*.

Retransmissão de um pacote (seq = 10) por falta de ACK

Figura 5: Execução do teste 5 com TIMEOUT = 2s e MAX_RETRIES = 3

O resultado deste teste demonstra exatamente o papel do Módulo **Reliability** a cumprir o seu propósito de controlo de falhas no protocolo Mission Link sob UDP. Dado que o Rover 3 é a comunicação mais desafiante (como vimos anteriormente) é de esperar o resultado observado: ambos os Rovers 1 e 2 conseguem trocar as mensagens relativas ao Teste nº5 (atribuição básica de missões aos 3 Rovers em simultâneo), já o terceiro Rover precisa de realizar a retransmissão de um pacote de um “MISSION PROGRESS” pois não recebeu o ACK da MotherShip dentro do tempo **RELIABILITY_TIMEOUT**, então retransmite.

Este comportamento é ainda mais evidente se definir esta variável de *timeout* igual a **0.1 segundos**. A próxima figura demonstra que os Rover 1 e 2 mesmo com algumas retransmissões conseguem proceder sem falhas na entrega, mas o Rover 3 têm dificuldades em receber os ACK's vindos da MotherShip excedendo as 3 tentativas provenientes de **RELIABILITY_MAX_RETRIES**:

Rover 3 atinge o limite de tentativas e denota falha na entrega do seq = 7

Figura 6: Execução do teste 5 com TIMEOUT = 0.1s e MAX_RETRIES = 3

Acrescentar também que as configurações dos caminhos descritos nesta secção, assim como os detalhes das ligações estão descritas no ficheiro `plan_types.txt`, ou no próprio ficheiro da topologia `TP2.xml`, para consulta.

4 . Conclusões Finais

Este projeto foi importante para comprovar, no simulador CORE, a resiliência dos protocolos MissionLink (UDP) e TelemetryStream (TCP) que foram implementados pelo grupo, sob várias condições de rede adversas como alta latência, *jitter* e perdas de pacotes. Os testes mostraram que:

- O mecanismo de fiabilidade implementado no MissionLink, que inclui numeração sequencial, ACKs, retransmissões e um timeout configurável, foi suficientemente robusto para recuperar de falhas de entrega, garantindo consistência mesmo em caso de perdas significativas.
- A comunicação da Telemetria periódica foi igualmente robusta, e o facto de ser em TCP garante que os dados sejam entregues na mesma ligação e que nenhuma parte da stream seja perdida, logo a monitorização num cenário como este é fiável e faz sentido neste contexto.
- Em situações extremas, especialmente para o rover que tinha o caminho de rede mais problemático (Rover 3), apesar de ser possível que a retransmissão incessante atrase as missões, o que acontece é que o comportamento é perfeitamente determinista e previsível, comprovando a consistente implementação desse mecanismo de controlo de falhas em UDP.
- A API da MotherShip mostrou-se rápida e consistente o suficiente para alimentar o Ground Control e assim proporcionar, à distância, uma janela para o estado interno, à semelhança do que acontece na realidade. Tivemos também oportunidade de explorar um tipo de comunicação que nos é muito comum que é a Comunicação Web, neste caso por WebSocket.

No geral, os resultados comprovam que o sistema é funcional, resiliente e capaz de lidar com falhas de comunicação moderadas a severas, mantendo o ciclo de vida das missões sem inconsistências.

4.1 Melhorias Futuras

Este sistema cumpre os requisitos do enunciado e funciona como esperado, mas existem melhorias que poderíamos adicionar e aqui ficam algumas que gostaríamos de adicionar se continuássemos com o desenvolvimento do projeto:

- *Timeout* e *Retries* dinamicamente: Ajustar automaticamente os parâmetros de confiabilidade com base na qualidade da rede observada, diminuindo retransmissões desnecessárias em bons caminhos e aumentando a tolerância em caminhos maus.
- Prioridade de Pacotes/Missões: Introduzir prioridades (ex.: ELEVADA, BAIXA, URGENTE, ...) para ajustar o comportamento em situações de congestionamento e de aproximação ao realismo.
- “Full-duplex” heartbeat com o tipo **HELLO**: uma ideia que poderia ser alargada para deteção de problemas de conexão e atualização de estados de conectividade.
- Compressão dinâmica do MissionLink: diminuir o tamanho dos payloads poderia diminuir o impacto da perda e jitter.
- Melhorar a simulação dos sistemas que implementamos: aspetos como a bateria e a temperatura, mas principalmente os sensores, poderiam ser melhor formulados para efeitos de realismo.
- Persistência no Ground Control: Historico longo, evolução e gráficos e *playback* de cenários.
- Várias MotherShips e mais Mecanismos de Redundância: para ainda mais fiabilidade e teste da robustez dos protocolos que consideramos lidarem bem com a concorrência.

Estas melhorias embora não sejam essenciais ao funcionamento do sistema de base poderiam por o nosso sistema num patamar mais proximo de sistemas reais usados em missões de exploracao remota e comunicacao em rede de computadores.